

DEEP LEARNING FOR FINANCE

MODULE 3 | LESSON 4

TRANSFORMING TIME SERIES INTO IMAGES

Reading Time	45 minutes
Prior Knowledge	Time series, algebra, MLPs
Keywords	Images, Time series

1. INTRODUCTION

The use of Convolutional Neural Networks has become the state-of-the-art approach in various image recognition, computer vision, and speech recognition tasks. This is due to their success in tackling complex problems thanks to their ability to extract relevant information from image-like data structures from certain location-invariance assumptions and weight sharing (See Module 2 of this course).

In the next two modules of this course, we will describe the main Deep Learning models that are used in time series prediction problems (Recurrent Neural Networks among other architectures). But before describing those concepts, it is worth mentioning that there are useful methodologies that allow us to transform time series information into images. Those image versions of time series can then be used as inputs to CNNs that allow us to predict the labels associated to each time series.

Transforming time series into images may turn out to be more effective than resorting to Deep Learning time series models due to two reasons. First, CNNs usually have better behavior while training. Second, we can exploit complex pre-trained CNN architectures, which are not readily available for explicit time series architectures.

In this notebook, we implement the Gramian Angular Field (GAF) methodology that we described in Lesson 3, and illustrate its outcome for a general financial time series. In the previous Lesson, we showed a straightforward coding implementation in Python for GAF. In this notebook, we are going to make use of the "pyts" library, which we load below and that includes a readily available class that performs the GAF transformation.

```
In [1]: import matplotlib.pyplot as plt
        from pyts.image import GramianAngularField
```

First, we show for a sample open-source dataset, [GunPoint](#), how GAF makes the transformations of time series. The information of this dataset, created for computer vision tasks, provides information about the location of an actor's hands in a sequence where the actor draws a gun (or pretends to draw a gun). The images are represented from a side view. Please, follow the previous link if you are further interested in learning about this dataset.

When we load the dataset, we obtain an two-dimensional array where, out of the 50 rows, each row contains information of a time series with 150 observations. Each time series is "contiguous" to each other, by shifting one time period the observations of the previous sequence. This means that the second row contains the same time series than the first row, except for the first and last observations. The first observation in the second row is the second observation of the first row, the second observation in the second row is the third observation of the first row and so on...

Formally, consider a time series with realizations $\{x_1, x_2, \dots, x_K\}$. We can split the time series into sequences of T observations $\{x_t, x_{t+1}, x_{t+2}, \dots, x_{t+T}\}$, for $t \in [1, K-T]$. Then, we form the array X that stacks each of these sequences:

$$X = \begin{pmatrix} x_1 & x_2 & \cdots & x_T & x_{T+1} & x_{T+2} & \cdots & x_{K-T} & x_{K-T+1} & \cdots & x_K \end{pmatrix}$$

```
In [2]: from pyts.datasets import load_gunpoint

X, _, _, _ = load_gunpoint(return_X_y=True)
# Check the shape of the array
print(X.shape)
# Print the first 10 observations of the first two rows
# Check that both sequences are contiguous to each other
print(X[0, 0:10])
print(X[0, 1:11])
```

```
(50, 150)
[-0.6478854 -0.64199155 -0.63818632 -0.63825875 -0.63834515 -0.63869741
 -0.64304876 -0.64376789 -0.64504991 -0.64711823]
[-0.64199155 -0.63818632 -0.63825875 -0.63834515 -0.63869741 -0.64304876
 -0.64376789 -0.64504991 -0.64711823 -0.64915334]
```

Now we transform each time series using the Gramian Angular Field. Recall that the transformation involves, first, a Min-Max transformation of each time series that rescales the information to the $[-1, 1]$ interval. Second, we compute the polar coordinates, importantly the term $\phi_t = \arccos(\tilde{x}_t)$, for each observation of the scaled time series. Finally, we build the Gram-like matrix, for each time series:

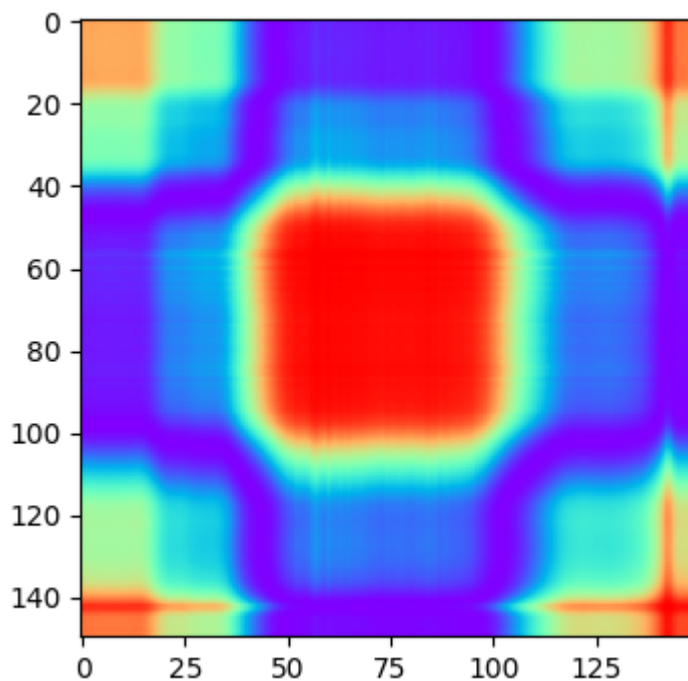
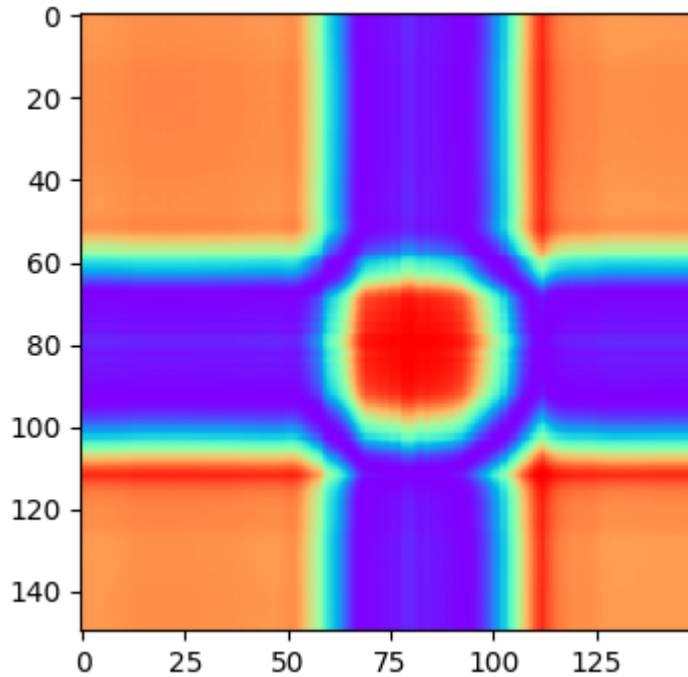
$$G = \begin{pmatrix} \cos(\phi_1 - \phi_1) & \cos(\phi_1 - \phi_2) & \cdots & \cos(\phi_1 - \phi_T) \\ \cos(\phi_2 - \phi_1) & \cos(\phi_2 - \phi_2) & \cdots & \cos(\phi_2 - \phi_T) \\ \vdots & \vdots & \ddots & \vdots \\ \cos(\phi_T - \phi_1) & \cos(\phi_T - \phi_2) & \cdots & \cos(\phi_T - \phi_T) \end{pmatrix}$$

```
In [3]: transformer = GramianAngularField()
X_new = transformer.transform(X)
```

```
# Show the results for the first time series
plt.figure(figsize=(4, 4))
plt.imshow(X_new[0], cmap="rainbow")
print(X_new[0].shape)
# Show the results for the 10th time series
plt.figure(figsize=(4, 4))
plt.imshow(X_new[9], cmap="rainbow")
print(X_new[9].shape)
```

(150, 150)

(150, 150)



The image version of the dataset can then be used to train a convolutional neural network designed to learn from an image if a person, from a side view, is drawing a gun or not.

3. GAF FOR A FINANCIAL TIME SERIES

Now, let's perform the same task for a financial time series. We are going to use daily information about the close price of the ETF QQQ that tracks the Nasdaq index. We use more than 20 years of information.

```
In [4]: import numpy as np
import pandas as pd

# Read the CSV file
file_path = 'QQQ.csv'
df = pd.read_csv(file_path, delimiter=';', decimal=',', parse_dates=['Date'], da
df = df.sort_values('Date')
df = df.set_index('Date')

# Rename '% Return' column to 'Ret'
df = df.rename(columns={'% Return': 'Ret'})

# Define the name variable
name = 'Ret'

# Set 'Date' as index and keep only 'Ret' column
df = df.dropna()

df.head()
```

Out[4]:

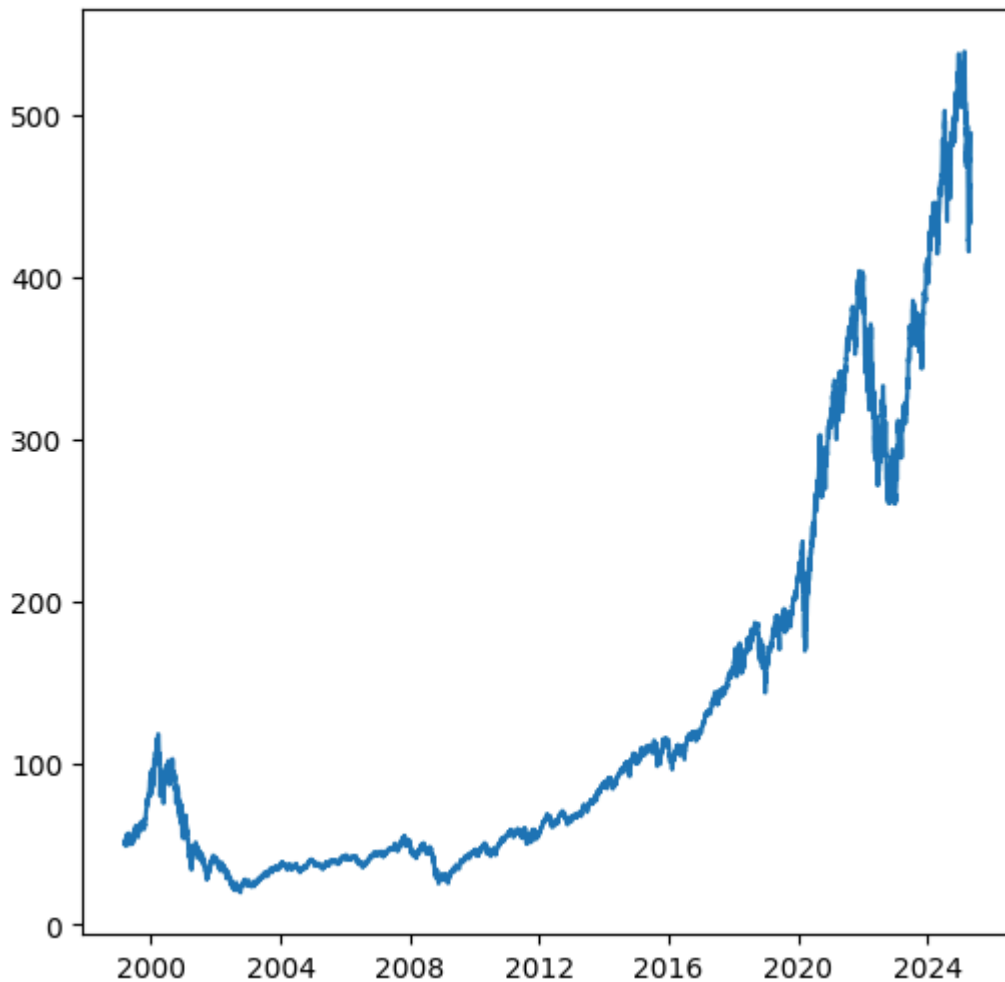
	Price	CVol	% Change	Open	Low	High	NAV	Total Return (Gross)	Ret	Cumulative Return %
Date										
1999-03-11	51.31	9688600.0	0.49	51.44	50.31	51.73	51.10	51.31	0.49	0,49
1999-03-12	50.06	8743600.0	-2.44	51.13	49.66	51.16	50.22	50.06	-2.44	-1,96
1999-03-15	51.50	6369000.0	2.87	50.44	49.91	51.56	51.51	51.50	2.87	0,86
1999-03-16	51.94	4905800.0	0.85	51.72	51.16	52.16	51.83	51.94	0.85	1,71
1999-03-17	51.56	3965000.0	-0.72	51.94	51.41	52.00	51.61	51.56	-0.72	0,98



```
In [5]: Xdf = df.iloc[:, 0]
print(Xdf.shape)
plt.figure(figsize=(6, 6))
plt.plot(Xdf)
```

(6548,)

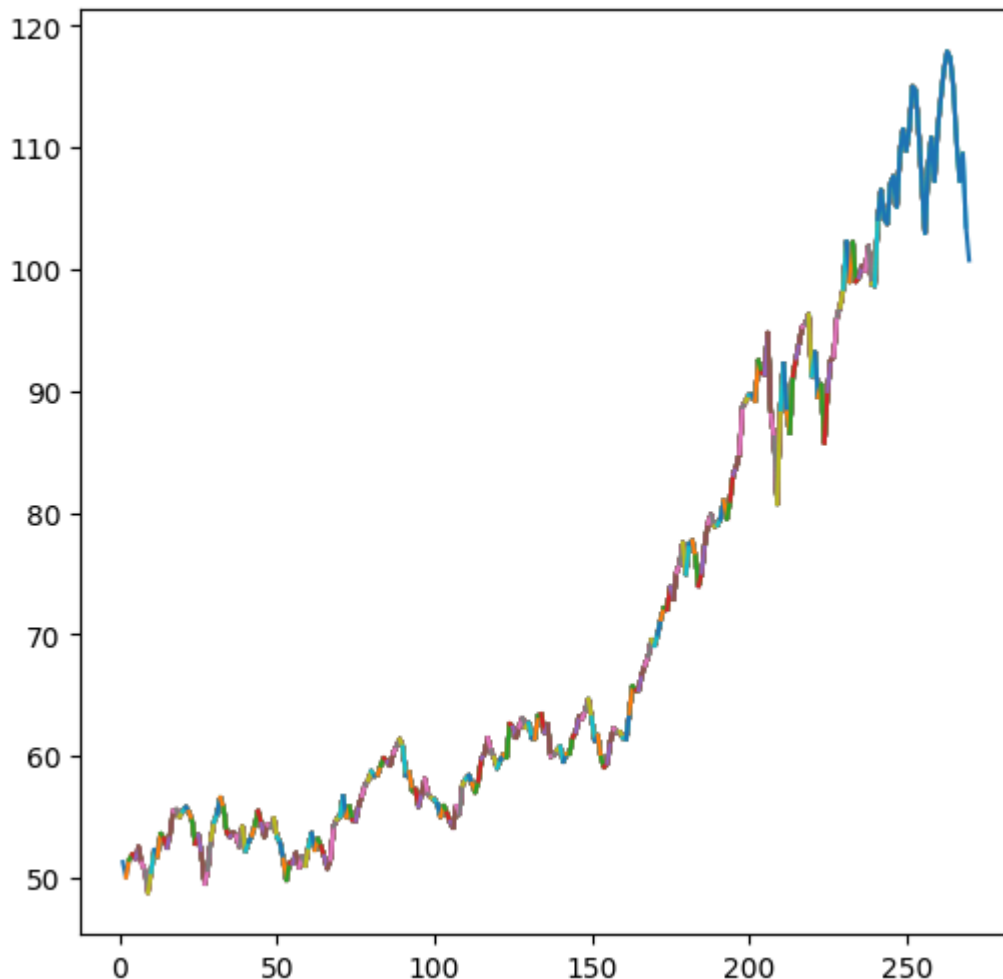
Out[5]: [



We now build the X matrix that we defined above for the new data, by stacking time series of 30 observations. We illustrate in the figure below how X represents information about 5,781 time series of 30 observations on overlapping intervals as we described above.

```
In [6]: X_train = []
window_size = 30
for i in range(window_size, Xdf.shape[0]):
    X_train.append(Xdf[i - window_size : i])
X_train = np.array(X_train)
print(X_train.shape)
# Define a "window_size"-days interval
days = np.linspace(1, X_train.shape[0], num=X_train.shape[0])
# Plot the overlapping time series
plt.figure(figsize=(6, 6))
for i in range(241):
    plt.plot(days[i : window_size + i], X_train[i, :])
```

(6518, 30)



We then apply the GAF transformation and illustrate the outcomes for some observations. Notice how the transformation gives values close to zero to those observations that are close to the average of the time series, while it gives high values to those realizations that are relatively more "extreme" within each time series.

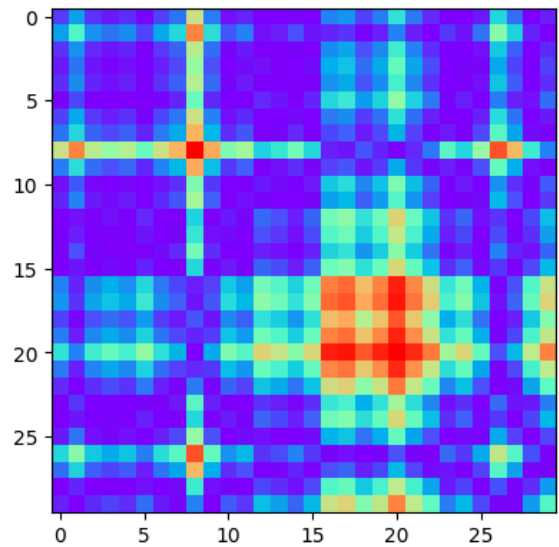
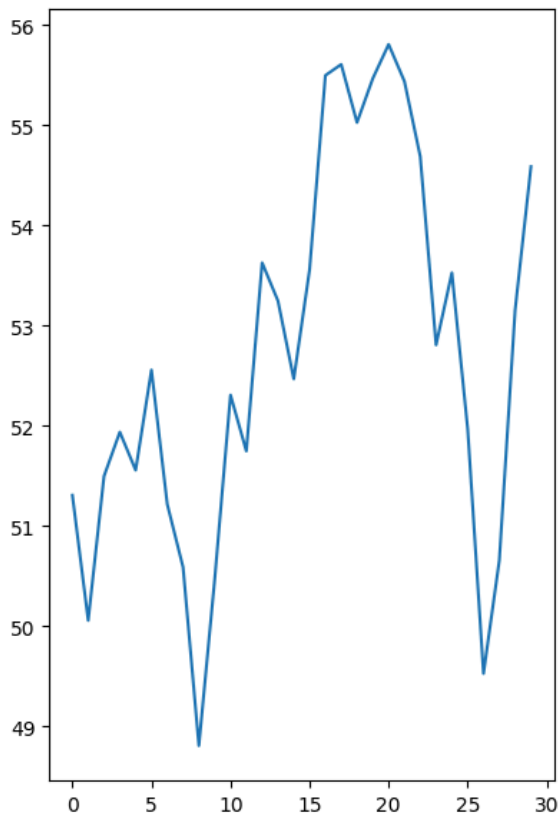
We now have a sample of images, to which we can attach a label, say, whether the price went up or down over the following days, and train a Convolutional Neural Network using the procedures we described in Module 2 and the first two lessons of Module 3.

```
In [7]: transformer = GramianAngularField()
X_train_new = transformer.transform(X_train)
print(X_train_new.shape)
```

```
(6518, 30, 30)
```

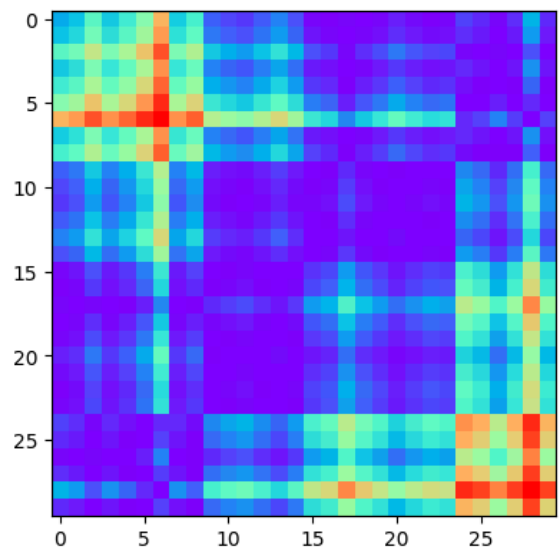
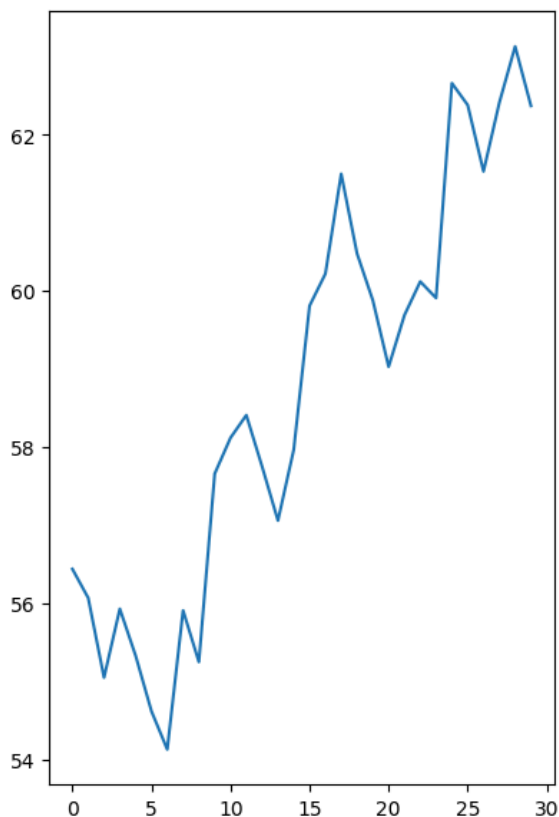
```
In [8]: # Show the results for the first time series
fig, ax = plt.subplots(1, 2, figsize=(10, 7), gridspec_kw={"width_ratios": [1, 1]})
ax[0].plot(X_train[0, :])
ax[1].imshow(X_train_new[0], cmap="rainbow")
print(X_train_new[0].shape)
```

```
(30, 30)
```



```
In [9]: # Show the results for the 100th time series
fig, ax = plt.subplots(1, 2, figsize=(10, 7), gridspec_kw={"width_ratios": [1, 1]})
ax[0].plot(X_train[99, :])
ax[1].imshow(X_train_new[99], cmap="rainbow")
print(X_train_new[99].shape)
```

(30, 30)



3. USING THE TRANSFORMED SERIES IN A CNN

In the blocks of code below, we design a convolutional neural network to predict the evolution of the time series. By now, you should be familiar with this type of structure, so we leave the analysis of the network architecture and its performance to your own judgement.

```
In [10]: import tensorflow as tf
```

```
2025-05-07 08:17:17.637008: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-05-07 08:17:17.675082: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2025-05-07 08:17:18.774566: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
```

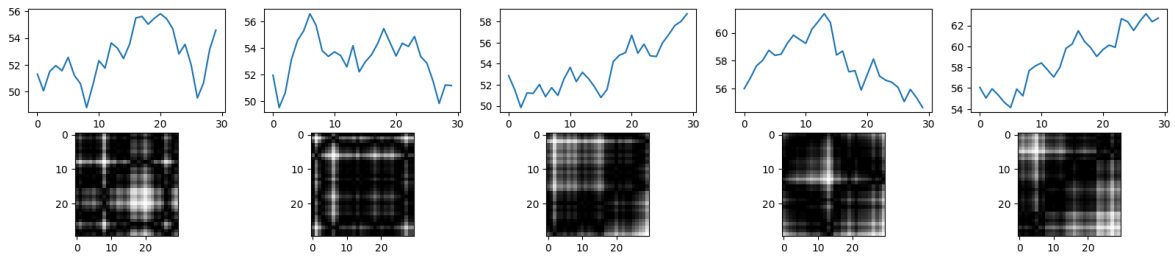
```
In [11]: # Generate the labels: 1 if price goes up after window_size days, 0 otherwise
y_data = []
for i in range(window_size, Xdf.shape[0] - window_size):
    y_data.append(Xdf[i] < Xdf[i + window_size])
y_data = np.array(y_data)
y_data.shape
```

```
Out[11]: (6488,)
```

```
In [12]: # Determine the examples to be used as inputs in the model and rescale them to t
X_data = X_train_new[:-window_size]
for i in range(X_data.shape[0]):
    X_data[i] = (X_data[i] - np.min(X_data[i])) / (
        np.max(X_data[i]) - np.min(X_data[i])
    )
# We could simply write X_data = (X_data + 1)/2
X_data = X_data * 255
X_data.shape
```

```
Out[12]: (6488, 30, 30)
```

```
In [13]: # display several time series and the corresponding images
fig, ax = plt.subplots(2, 5, figsize=(20, 4))
for i in range(0, 125, 25):
    ax[0, int(i / 25)].plot(X_train[i])
for i in range(0, 125, 25):
    ax[1, int(i / 25)].imshow(X_data[i], "gray")
```



```
In [14]: # Split into train and test
train_split = 0.8
y_data_train = y_data[: int(train_split * y_data.shape[0])]
y_data_test = y_data[int(train_split * y_data.shape[0]) :]
X_data_train = X_data[: int(train_split * y_data.shape[0])]
X_data_test = X_data[int(train_split * y_data.shape[0]) :]
```

```
In [15]: model = tf.keras.Sequential()
# Input Layer
model.add(tf.keras.layers.InputLayer(input_shape=(30, 30, 1)))
model.add(tf.keras.layers.Conv2D(16, 3, activation="relu"))
model.add(tf.keras.layers.MaxPooling2D(2))
model.add(tf.keras.layers.Conv2D(32, 3, activation="relu"))
model.add(tf.keras.layers.MaxPooling2D(2))
model.add(tf.keras.layers.Conv2D(64, 3, activation="relu"))
model.add(tf.keras.layers.MaxPooling2D(2))
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(1024, activation="relu"))
model.add(tf.keras.layers.Dropout(0.5, seed=1234))
model.add(tf.keras.layers.Dense(1, activation="sigmoid"))
```

```
/usr/local/lib/python3.11/site-packages/keras/src/layers/core/input_layer.py:27:
UserWarning: Argument `input_shape` is deprecated. Use `shape` instead.
warnings.warn(
```

```
In [16]: model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 16)	160
max_pooling2d (MaxPooling2D)	(None, 14, 14, 16)	0
conv2d_1 (Conv2D)	(None, 12, 12, 32)	4,640
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 32)	0
conv2d_2 (Conv2D)	(None, 4, 4, 64)	18,496
max_pooling2d_2 (MaxPooling2D)	(None, 2, 2, 64)	0
flatten (Flatten)	(None, 256)	0
dense (Dense)	(None, 1024)	263,168
dropout (Dropout)	(None, 1024)	0
dense_1 (Dense)	(None, 1)	1,025



Total params: 287,489 (1.10 MB)

Trainable params: 287,489 (1.10 MB)

Non-trainable params: 0 (0.00 B)

```
In [17]: model.compile(loss="binary_crossentropy", optimizer="RMSprop", metrics=["accuracy",
checkpoint = tf.keras.callbacks.ModelCheckpoint(
    "model.h5", monitor="val_accuracy", save_best_only=True, mode="auto"
)
class_weight = {0: 1 / (1 - np.mean(y_data_train)), 1: 1.0}
print(class_weight)
```

```
{0: 2.694704049844237, 1: 1.0}
```

```
In [18]: model.fit(
    x=X_data_train,
    y=y_data_train,
    epochs=50,
    validation_data=(X_data_test, y_data_test),
    verbose=1,
    callbacks=[checkpoint],
    class_weight=class_weight,
)
```

Epoch 1/50

157/163 — 0s 6ms/step - accuracy: 0.4493 - loss: 4.6532

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.save\_model(model, 'my\_model.keras')`.

163/163  2s 8ms/step - accuracy: 0.4479 - loss: 4.5362 - val_accuracy: 0.3567 - val_loss: 0.7583

Epoch 2/50

163/163  2s 7ms/step - accuracy: 0.3891 - loss: 1.1232 - val_accuracy: 0.3536 - val_loss: 0.7725

Epoch 3/50

161/163  0s 5ms/step - accuracy: 0.3793 - loss: 1.0928

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

163/163  1s 7ms/step - accuracy: 0.3793 - loss: 1.0929 - val_accuracy: 0.4076 - val_loss: 0.7604

Epoch 4/50

163/163  1s 7ms/step - accuracy: 0.3914 - loss: 1.0963 - val_accuracy: 0.3567 - val_loss: 0.8211

Epoch 5/50

163/163  1s 6ms/step - accuracy: 0.3860 - loss: 1.0964 - val_accuracy: 0.3582 - val_loss: 0.7938

Epoch 6/50

163/163  1s 7ms/step - accuracy: 0.3849 - loss: 1.0875 - val_accuracy: 0.3629 - val_loss: 0.8184

Epoch 7/50

163/163  1s 7ms/step - accuracy: 0.4156 - loss: 1.0927 - val_accuracy: 0.3659 - val_loss: 0.7806

Epoch 8/50

163/163  1s 7ms/step - accuracy: 0.4160 - loss: 1.0913 - val_accuracy: 0.3775 - val_loss: 0.7931

Epoch 9/50

163/163  1s 7ms/step - accuracy: 0.4063 - loss: 1.0803 - val_accuracy: 0.3844 - val_loss: 0.7909

Epoch 10/50

163/163  1s 7ms/step - accuracy: 0.4369 - loss: 1.0790 - val_accuracy: 0.3636 - val_loss: 0.8089

Epoch 11/50

163/163  1s 7ms/step - accuracy: 0.4382 - loss: 1.0683 - val_accuracy: 0.3683 - val_loss: 0.8532

Epoch 12/50

163/163  1s 7ms/step - accuracy: 0.4492 - loss: 1.0550 - val_accuracy: 0.3898 - val_loss: 0.9225

Epoch 13/50

163/163  1s 7ms/step - accuracy: 0.4702 - loss: 1.0275 - val_accuracy: 0.3968 - val_loss: 0.8909

Epoch 14/50

156/163  0s 6ms/step - accuracy: 0.5021 - loss: 1.0179

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```

163/163 ————— 1s 7ms/step - accuracy: 0.5014 - loss: 1.0182 - val_
accuracy: 0.4222 - val_loss: 0.7789
Epoch 15/50
163/163 ————— 1s 6ms/step - accuracy: 0.5141 - loss: 0.9913 - val_
accuracy: 0.3991 - val_loss: 0.8495
Epoch 16/50
163/163 ————— 1s 7ms/step - accuracy: 0.5258 - loss: 0.9868 - val_
accuracy: 0.3852 - val_loss: 0.9068
Epoch 17/50
163/163 ————— 1s 6ms/step - accuracy: 0.5388 - loss: 0.9629 - val_
accuracy: 0.4153 - val_loss: 0.9118
Epoch 18/50
163/163 ————— 1s 7ms/step - accuracy: 0.5681 - loss: 0.9237 - val_
accuracy: 0.3806 - val_loss: 1.0185
Epoch 19/50
158/163 ————— 0s 5ms/step - accuracy: 0.5816 - loss: 0.9085
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
163/163 ————— 1s 7ms/step - accuracy: 0.5816 - loss: 0.9086 - val_
accuracy: 0.4638 - val_loss: 0.9787
Epoch 20/50
163/163 ————— 1s 7ms/step - accuracy: 0.6058 - loss: 0.8744 - val_
accuracy: 0.3875 - val_loss: 1.1807
Epoch 21/50
163/163 ————— 1s 7ms/step - accuracy: 0.6040 - loss: 0.8703 - val_
accuracy: 0.4422 - val_loss: 0.9922
Epoch 22/50
161/163 ————— 0s 6ms/step - accuracy: 0.6270 - loss: 0.8348
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
163/163 ————— 1s 7ms/step - accuracy: 0.6270 - loss: 0.8351 - val_
accuracy: 0.4723 - val_loss: 1.3726
Epoch 23/50
163/163 ————— 1s 7ms/step - accuracy: 0.6421 - loss: 0.8420 - val_
accuracy: 0.4060 - val_loss: 1.1111
Epoch 24/50
163/163 ————— 1s 7ms/step - accuracy: 0.6653 - loss: 0.7827 - val_
accuracy: 0.4106 - val_loss: 1.2552
Epoch 25/50
160/163 ————— 0s 5ms/step - accuracy: 0.6644 - loss: 0.7699
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
163/163 ————— 1s 7ms/step - accuracy: 0.6644 - loss: 0.7704 - val_
accuracy: 0.4823 - val_loss: 1.4214
Epoch 26/50
163/163 ————— 1s 7ms/step - accuracy: 0.7024 - loss: 0.7693 - val_
accuracy: 0.4669 - val_loss: 1.4488
Epoch 27/50
163/163 ————— 1s 6ms/step - accuracy: 0.7185 - loss: 0.7382 - val_
accuracy: 0.4122 - val_loss: 1.4385
Epoch 28/50
159/163 ————— 0s 5ms/step - accuracy: 0.7241 - loss: 0.6966

```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

163/163  1s 7ms/step - accuracy: 0.7241 - loss: 0.6966 - val_accuracy: 0.4938 - val_loss: 1.9754

Epoch 29/50

163/163  1s 6ms/step - accuracy: 0.7533 - loss: 0.6517 - val_accuracy: 0.4492 - val_loss: 1.1260

Epoch 30/50

163/163  1s 7ms/step - accuracy: 0.7464 - loss: 0.6854 - val_accuracy: 0.4507 - val_loss: 1.5861

Epoch 31/50

163/163  1s 7ms/step - accuracy: 0.7698 - loss: 0.6321 - val_accuracy: 0.3690 - val_loss: 1.3616

Epoch 32/50

156/163  0s 7ms/step - accuracy: 0.7482 - loss: 0.6376

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

163/163  1s 8ms/step - accuracy: 0.7491 - loss: 0.6366 - val_accuracy: 0.5316 - val_loss: 1.2804

Epoch 33/50

163/163  2s 7ms/step - accuracy: 0.7852 - loss: 0.6244 - val_accuracy: 0.4530 - val_loss: 2.2426

Epoch 34/50

163/163  1s 6ms/step - accuracy: 0.7879 - loss: 0.5942 - val_accuracy: 0.4707 - val_loss: 1.7875

Epoch 35/50

163/163  1s 6ms/step - accuracy: 0.8056 - loss: 0.5323 - val_accuracy: 0.4414 - val_loss: 1.9539

Epoch 36/50

159/163  0s 5ms/step - accuracy: 0.8079 - loss: 0.5559

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```

163/163 ————— 1s 6ms/step - accuracy: 0.8081 - loss: 0.5555 - val_
accuracy: 0.5986 - val_loss: 4.9630
Epoch 37/50
163/163 ————— 1s 7ms/step - accuracy: 0.8286 - loss: 0.5641 - val_
accuracy: 0.5670 - val_loss: 2.7113
Epoch 38/50
163/163 ————— 1s 6ms/step - accuracy: 0.8182 - loss: 0.5301 - val_
accuracy: 0.5069 - val_loss: 1.8596
Epoch 39/50
163/163 ————— 1s 7ms/step - accuracy: 0.8382 - loss: 0.4852 - val_
accuracy: 0.4430 - val_loss: 2.3383
Epoch 40/50
163/163 ————— 1s 7ms/step - accuracy: 0.8498 - loss: 0.4576 - val_
accuracy: 0.5331 - val_loss: 2.4713
Epoch 41/50
163/163 ————— 1s 7ms/step - accuracy: 0.8651 - loss: 0.4176 - val_
accuracy: 0.4214 - val_loss: 2.8916
Epoch 42/50
163/163 ————— 1s 7ms/step - accuracy: 0.8433 - loss: 0.4399 - val_
accuracy: 0.4561 - val_loss: 1.9956
Epoch 43/50
163/163 ————— 1s 7ms/step - accuracy: 0.8592 - loss: 0.4387 - val_
accuracy: 0.5424 - val_loss: 2.2146
Epoch 44/50
163/163 ————— 1s 7ms/step - accuracy: 0.8759 - loss: 0.4071 - val_
accuracy: 0.5254 - val_loss: 2.3211
Epoch 45/50
163/163 ————— 1s 7ms/step - accuracy: 0.8796 - loss: 0.3935 - val_
accuracy: 0.5478 - val_loss: 2.5184
Epoch 46/50
163/163 ————— 1s 7ms/step - accuracy: 0.8987 - loss: 0.3674 - val_
accuracy: 0.4892 - val_loss: 2.1071
Epoch 47/50
163/163 ————— 1s 7ms/step - accuracy: 0.8787 - loss: 0.3893 - val_
accuracy: 0.5116 - val_loss: 2.2406
Epoch 48/50
163/163 ————— 1s 7ms/step - accuracy: 0.9036 - loss: 0.3148 - val_
accuracy: 0.4992 - val_loss: 2.4010
Epoch 49/50
163/163 ————— 1s 6ms/step - accuracy: 0.8999 - loss: 0.3332 - val_
accuracy: 0.5162 - val_loss: 2.5038
Epoch 50/50
163/163 ————— 1s 7ms/step - accuracy: 0.9013 - loss: 0.3538 - val_
accuracy: 0.4977 - val_loss: 2.3817

```

Out[18]: <keras.src.callbacks.history.History at 0x7b03a87ed950>

```

In [19]: model = tf.keras.models.load_model("model.h5")
acc = model.evaluate(x=X_data_test, y=y_data_test)
print("Accuracy over validation: {:.2f}%".format(acc[1] * 100))

```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until you train or evaluate the model.

```

41/41 ————— 0s 2ms/step - accuracy: 0.6015 - loss: 3.7747
Accuracy over validation: 59.86%

```

```

In [20]: from sklearn import metrics
from sklearn.metrics import classification_report

```

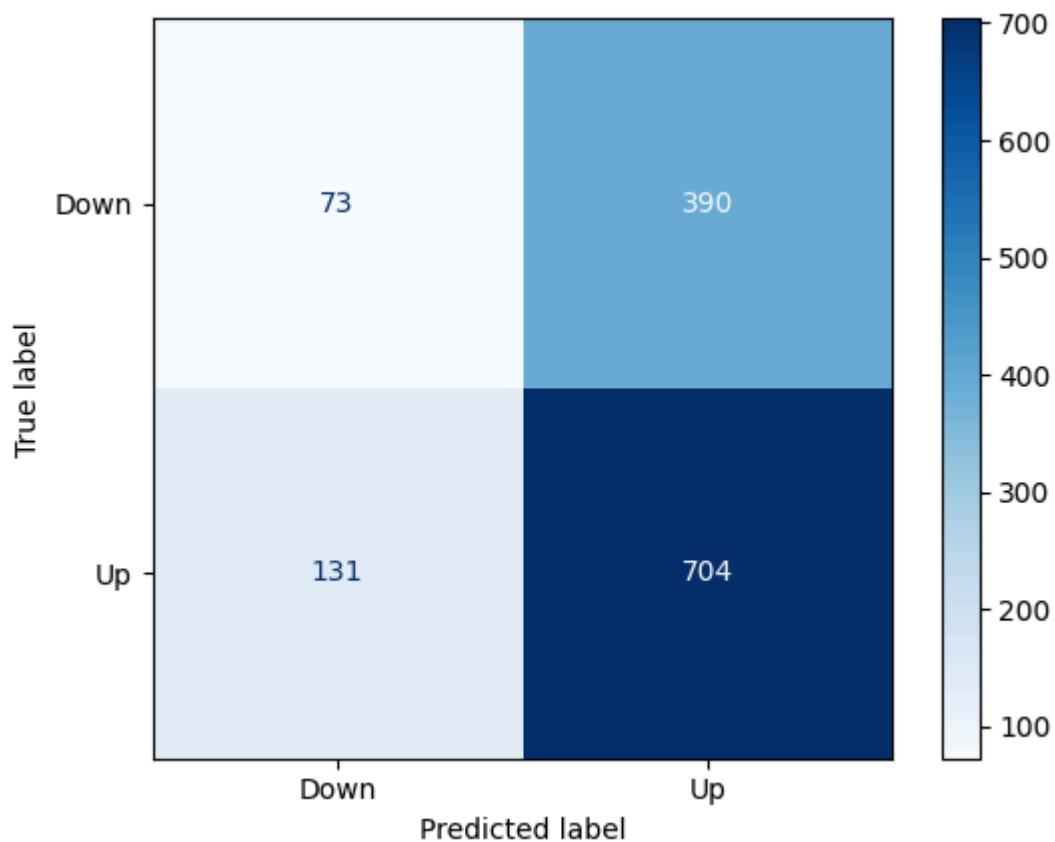
```

y_prob = model.predict(X_data_test)
y_pred = np.round(y_prob)
cm = metrics.confusion_matrix(y_data_test, y_pred)
cm = metrics.ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Down",
cm.plot(values_format="", cmap="Blues")
print(classification_report(y_data_test, y_pred))

```

41/41 ————— 0s 3ms/step

	precision	recall	f1-score	support
False	0.36	0.16	0.22	463
True	0.64	0.84	0.73	835
accuracy			0.60	1298
macro avg	0.50	0.50	0.47	1298
weighted avg	0.54	0.60	0.55	1298



4. CONCLUSIONS

In this lesson, we have described the implementation of the GAF transformation to obtain images from time series information. This type of methodology may be effective to enhance our prediction tasks from time series models. Convolutional neural networks have been successful at performing challenging image recognition tasks due to their ability to extract relevant features from images. Thus, it may then be convenient to take advantage of this powerful tool, even when our data is not shown in the form of an image.

In Module 4, we will describe Deep Learning methods that are explicitly designed for the analysis of time series and sequence data.

See you there!

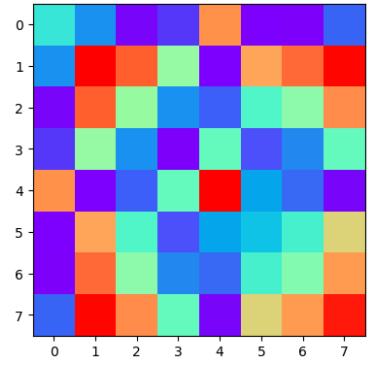
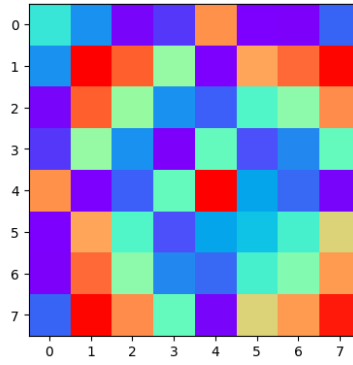
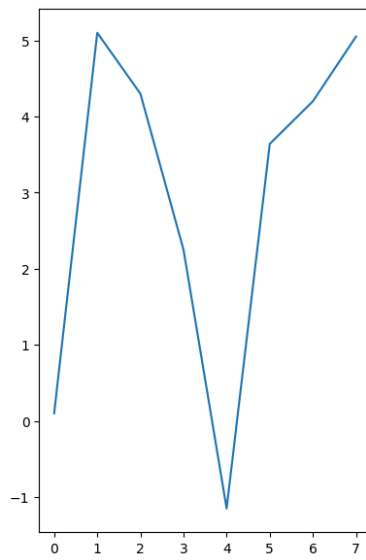
TESTS

Below we show the simple implementation of the GAF algorithm in a few lines of code and compare the results with the class included in the pyts library.

```
In [21]: XX = np.array([0.1, 5.1, 4.3, 2.25, -1.15, 3.64, 4.2, 5.05]) # X_train[99,:] #
# Manual transformation
XX_norm = (XX - np.min(XX)) / (np.max(XX) - np.min(XX))
XX_norm = XX_norm * 2 - 1
print(XX_norm)
XX_arccos = np.arccos(XX_norm)
print(XX_arccos)
XX_arccos_m = np.add.outer(XX_arccos, XX_arccos)
GramAngular = np.cos(XX_arccos_m)
print(GramAngular)
# Using the Library
GramAngular_pyts = transformer.transform(XX.reshape(1, -1))
# Show the results, compare the "from scratch" method with the pyts method
fig, ax = plt.subplots(1, 3, figsize=(15, 7), gridspec_kw={"width_ratios": [1, 1, 1]})
ax[0].plot(XX)
ax[1].imshow(GramAngular, cmap="rainbow")
ax[2].imshow(GramAngular_pyts[0], cmap="rainbow")
```

```
[ -0.6      1.      0.744  0.088 -1.      0.5328  0.712  0.984 ]
[ 2.21429744 0.      0.73175934 1.48268235 3.14159265 1.00889045
 0.77845394 0.17912481]
[ [-0.28      -0.6      -0.98094369 -0.84969638  0.6      -0.99667296
  -0.98894357 -0.73293477]
  [-0.6      1.      0.744      0.088      -1.      0.5328
   0.712      0.984      ]
  [-0.98094369 0.744      0.107072 -0.6001154 -0.744      -0.16903792
   0.06054599 0.61304747]
  [-0.84969638 0.088      -0.6001154 -0.984512 -0.088      -0.79607178
  -0.63679934 -0.09088525]
  [ 0.6      -1.      -0.744      -0.088      1.      -0.5328
  -0.712      -0.984      ]
  [-0.99667296 0.5328      -0.16903792 -0.79607178 -0.5328      -0.43224832
  -0.21485959 0.37350171]
  [-0.98894357 0.712      0.06054599 -0.63679934 -0.712      -0.21485959
   0.013888   0.57550177]
  [-0.73293477 0.984      0.61304747 -0.09088525 -0.984      0.37350171
   0.57550177 0.936512  ]]
```

```
Out[21]: <matplotlib.image.AxesImage at 0x7b03a12acad0>
```



Copyright 2025 WorldQuant University. This content is licensed solely for personal use.
Redistribution or publication of this material is strictly prohibited.

Module 3: Lesson 1

Deep Convolutional Neural Networks: Architectures



Outline

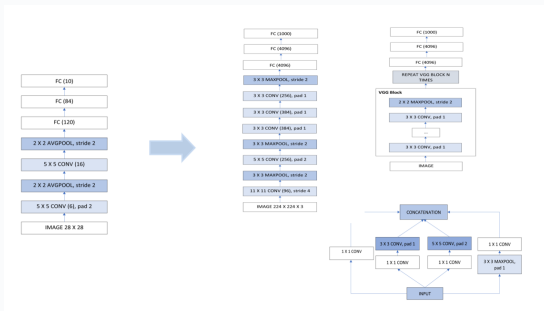
- ▶ LeNet and AlexNet
- ▶ Networks using blocks (VGG) and Network-in-Network (NiN)
- ▶ Multi-Branch networks (GoogLeNet)
- ▶ Residual networks (ResNet, ResNeXt, and DenseNet)
- ▶ From feature engineering to network engineering

CNN Architectures

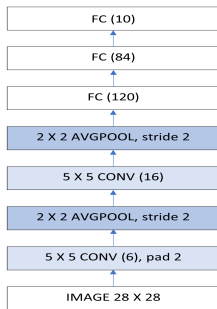
In Module 2, we described all the elements that are necessary to assemble and train a fully operational convolutional neural network.

In Module 3, we are going to delve into additional tools that may enhance the performance of CNNs.

This first lesson describes how new CNN architectures have been proposed and have evolved over time to tackle image recognition tasks.



LeNet



LeNet (LeCun et al., 1998) was one of the first CNNs successfully proposed for image recognition tasks, over other supervised learning tools.

The architecture was designed to process low-resolution images of handwritten digits.

The model combines two layers of 5x5 convolutions with pooling and three fully connected layers.

Originally, it used softmax activations and average pooling.

- LeNet was proposed even before ReLU and max-pooling were “discovered”!

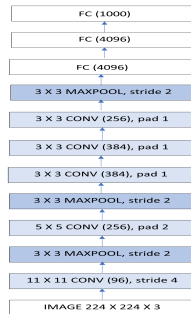
AlexNet

AlexNet (Krizhevsky et al., 2012) can be labeled as the first “modern” CNN.

This network was the first to show that learning algorithms can automatically learn features of images, transcending manually designed features.

New developments in GPU processing reduced the computational burden of deep learning architectures, which sparked their popularity.

AlexNet consists of eight layers, employs ReLU activations, and adds dropout in the fully-connected layers.



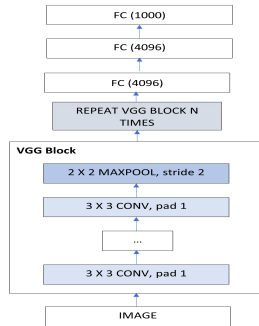
Networks using blocks (VGG)

After AlexNet, the design of neural network architectures has moved to thinking in terms of blocks that repeat patterns of layers.

- The VGG network introduced the preference for deep (many layers) against wide (many channels) networks.

A VGG block (Simonyan & Zisserman, 2014) consists of a sequence of convolutions with 3x3 kernels with padding of 1 (keeping height and width) followed by a 2x2 max-pooling layer with stride of 2 (halving height and width after each block).

The idea is to apply multiple convolutions and avoid a too-quick reduction in the spatial resolution.



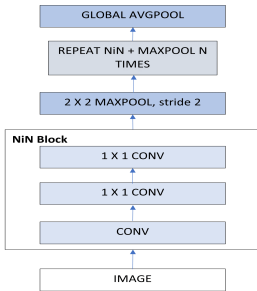
Network-in-Network

One of the major drawbacks of the previous architectures arises from the fully-connected layers that require large amounts of memory.

A Network-in-Network (NiN, Lin et al., 2013) block consists of a typical convolution followed by 1×1 convolutions, which can be thought of as fully connected layers acting independently on each pixel location.

The NiN network gets rid of the giant fully connected layers at the end of the network, which are replaced by a global average-pooling layer.

- This means that the last block must have as many output channels as the number of label classes.



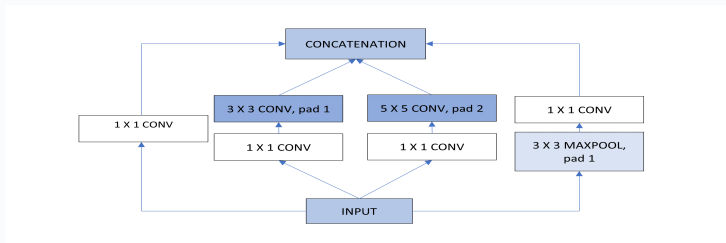
Multi-branch networks (GoogLeNet)

GoogLeNet (Szegedy et al., 2015) combines the strengths of repeated convolutions, NiN, and blocks.

GoogLeNet uses blocks of code (“Inception” blocks) that use four parallel branches.

The Inception blocks explore the image in a variety of filter sizes: Details at different extents can be recognized efficiently by filters of different sizes and different parameters.

- In Lesson 2, we will work on a specific version of this architecture, labeled “Xception.”



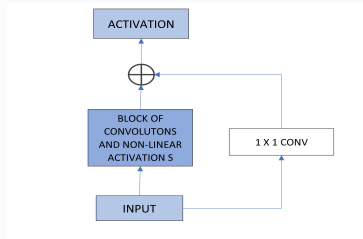
ResNet

When training a neural network, our aim is to approximate a general function $f(\mathbf{x})$ that allows the prediction of the output, given the inputs $\mathbf{x}$.

Similar to a Taylor series expansion, we can obtain an approximation $f(\mathbf{x}) = \mathbf{x} + g(\mathbf{x})$.

The ResNet (Residual Network, He et al., 2016) architecture aims at learning separately the identity mapping, $\mathbf{x}$ and the *residual mapping* $g(\mathbf{x}) = f(\mathbf{x}) - \mathbf{x}$.

In ResNet, a shortcut connection bypasses several convolutional layers, which deal with learning the non-linearities. Both components are then added to obtain the output.



ResNext and DenseNet

ResNet can be understood as a particular case of the Inception block. ResNeXt (Xie et al., 2017) generalizes the ResNet architecture to an Inception block that adopts the same transformation in all the “inside” branches.

At the end of each block of ResNet, the linear and the residual terms are added to obtain an output.

DenseNet, instead, concatenates each output of the residual block, which is fed separately into deeper layers of the network.

Each dense block will increase the number of channels, leading to an excessively complex model. A *transition layer* is used to control this.

From feature engineering to network engineering

Summary of the recent developments in the design of CNNs:

- ▶ AlexNet: Automatic feature extraction with deep CNNs.
- ▶ VGG Networks: Stacking of 3x3 convolutions improve the results of wide layers.
- ▶ NiN: Adding local non-linearities.
- ▶ GoogLeNet, ResNet, ResNeXt: Multi-branch networks that extract different components to approximate general functions.

This has led to new models where the design of the network is also trained on top of the usual model parameters in a neural network: AnyNet.

Summary of Lesson 1

In Lesson 1, we have:

- Described the evolution of CNN architectures over time, from LeNet to ResNext and AnyNet.

⇒ **References for this lesson:**

Chapters 7 & 8 in [Zhang, Aston, et al. *Dive into Deep Learning*. 2022.](#)

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will describe and implement transfer learning using another CNN architecture: Xception.

Module 3: Lesson 2

Deep Convolutional Neural Networks: Transfer Learning for CNNs



Outline

- ▶ Transfer Learning
- ▶ Xception
- ▶ Application: Stock return prediction with candlestick charts

Transfer Learning

Many CNN architectures have been proven successful at performing image recognition tasks.

- ▶ These architectures are usually large-scale models trained with large-scale datasets.

Transfer Learning: We can leverage those trained models and apply them to our, possibly smaller-scale, applications.

- ▶ The learning ability of the pre-trained model may capture general features of images, which may prove effective for recognizing the objects in our target dataset.

Steps in Transfer Learning:

1. Obtain the parameters of a pre-trained model from a source dataset.
2. Create the target model, copying the structure and parameter values of the pre-trained network, except the output layers.
3. Add output layers to the target model, adapted to the labels of our target database.
4. Train the target model on the target dataset. The pre-trained model parameters can be fine-tuned or kept fixed.

Xception: Separable convolutions

Xception is a CNN architecture that has been proposed as a generalization of Inception.

The architecture replaces the Inception blocks by layers of “separable convolutions.”

Separable convolutions involve two steps:

1. Apply a convolution operator to each input channel using a single kernel.
2. Apply 1×1 convolutions to the tensors that result from the previous steps. The number of 1×1 kernels, with a dimension equal to the number of input channels, must equal the number of output channels.

This operator reduces considerably the number of operations that are involved in a standard convolution operator.

The intuition follows VGG models: Deep networks are preferred over wide networks. Separable convolutions reduce the computational costs of increasing the depth of the architecture.

Xception: Example of separable convolution (First step)

INPUT		KERNEL		OUTPUT																	
<table><tr><td>1</td><td>2</td><td>4</td></tr><tr><td>3</td><td>7</td><td>2</td></tr><tr><td>2</td><td>0</td><td>2</td></tr></table>	1	2	4	3	7	2	2	0	2	*	<table><tr><td>3</td><td>2</td></tr><tr><td>2</td><td>1</td></tr></table>	3	2	2	1	=	<table><tr><td>20</td><td>30</td></tr><tr><td>27</td><td>27</td></tr></table>	20	30	27	27
1	2	4																			
3	7	2																			
2	0	2																			
3	2																				
2	1																				
20	30																				
27	27																				
<table><tr><td>0</td><td>2</td><td>1</td></tr><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>2</td><td>4</td><td>0</td></tr></table>	0	2	1	1	2	3	2	4	0	*	<table><tr><td>1</td><td>2</td></tr><tr><td>0</td><td>3</td></tr></table>	1	2	0	3	=	<table><tr><td>10</td><td>13</td></tr><tr><td>17</td><td>8</td></tr></table>	10	13	17	8
0	2	1																			
1	2	3																			
2	4	0																			
1	2																				
0	3																				
10	13																				
17	8																				
<table><tr><td>4</td><td>2</td><td>1</td></tr><tr><td>5</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr></table>	4	2	1	5	1	0	1	0	1	*	<table><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>2</td></tr></table>	0	1	1	2	=	<table><tr><td>9</td><td>2</td></tr><tr><td>2</td><td>2</td></tr></table>	9	2	2	2
4	2	1																			
5	1	0																			
1	0	1																			
0	1																				
1	2																				
9	2																				
2	2																				

Remember that in the standard convolution operator we would have as many kernel elements as output channels, each with dimension kernel height x kernel width x input channels.

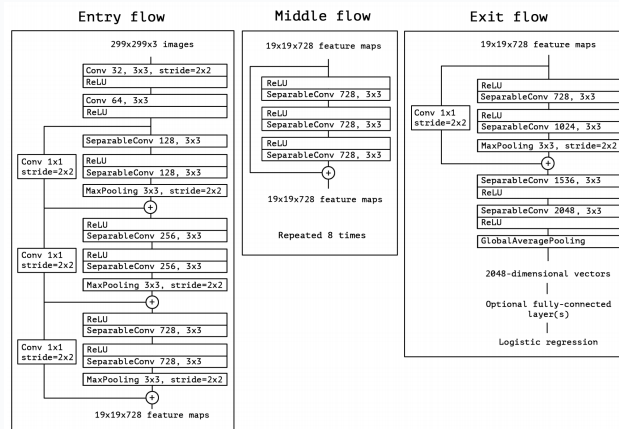
- In separable convolutions, we have just a single kernel element in the first step of computations.

Xception: Example of separable convolution (Second step)

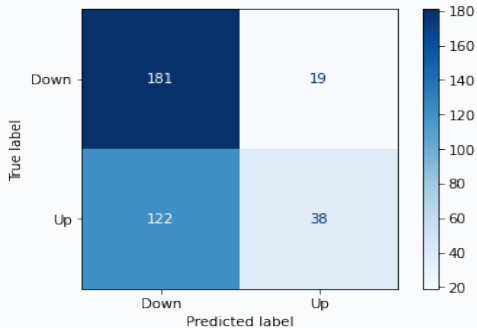
INPUT			KERNEL		OUTPUT
20	30	*	-1		
27	27			+	
10	13	*	1	=	8 -13
17	8				-6 -15
9	2	*	2		
2	2				

We repeat the above operations with as many 1x1 kernels as output channels we have in the layer.

Xception: Architecture



Application: Stock return prediction with candlestick charts



We train the candlestick-based stock return prediction model using transfer learning and the Xception architecture.

After training the model several times, using transfer learning leads to better performance than the basic CNN architecture of Module 2.

However, the model still seems to predict too often a "down" move and displays overfitting.

Summary of Lesson 2

In Lesson 2, we have:

- ▶ Described transfer learning as a useful tool to train CNNs
- ▶ Described the Xception CNN architecture
- ▶ Applied transfer learning and Xception to predict stock returns using candlestick charts

⇒ **References for this lesson:**

Chapter 8 and Section 14.2 in [Zhang, Aston, et al. *Dive into Deep Learning*. 2022.](#)

TO DO NEXT: Please, go to the Jupyter Notebook associated with Lesson 2.

In the next lesson, we'll describe additional tools to enhance the performance of CNN architectures.

Module 3: Lesson 3

Deep Convolutional Neural Networks: Encoding Time Series as Images



Outline

- ▶ From time series to images
- ▶ Gramian Angular Field (GAF) imaging
- ▶ A simple example of GAF

From time series to images

The success of Deep Learning largely arises due to the advances in computer vision and speech recognition.

Time series forecasting with explicitly designed models can be challenging.

- ▶ We'll cover Deep Learning architecture for time series in Modules 4 and 5 of this course.
- ▶ We cannot take advantage of pre-trained models.

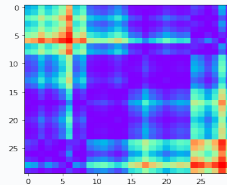
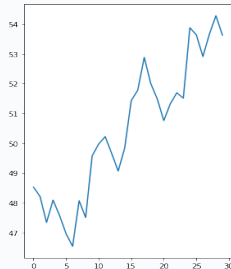
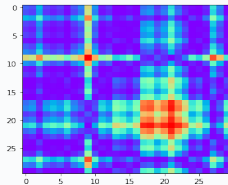
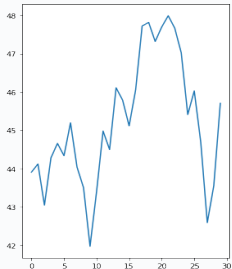
A shortcut to improving the performance of time series models may be to use convolutional neural networks.

But to do this, we need to develop a tool that transforms time series information into an image-like structure.

Here, we describe one way to encode time series as images: **Gramian Angular Field**.

Gramian Angular Field Imaging

With the Gramian Angular Field, we'll be able to represent a simple time series as an image that represents the features of the time series.



Math for Gramian Angular Field: Gram Matrix

The gram matrix is used to compute the extent of linear dependence between a set of vectors.

The gram matrix, G , of a set of n Euclidean vectors $\{v_1, \dots, v_n\}$, $v_i \in \mathbb{R}^k$, is defined by the dot-product of every pair of vectors in the set:

$$G = \begin{pmatrix} \langle v_1, v_1 \rangle & \langle v_1, v_2 \rangle & \cdots & \langle v_1, v_n \rangle \\ \langle v_2, v_1 \rangle & \langle v_2, v_2 \rangle & \cdots & \langle v_2, v_n \rangle \\ \vdots & \vdots & \ddots & \vdots \\ \langle v_n, v_1 \rangle & \langle v_n, v_2 \rangle & \cdots & \langle v_n, v_n \rangle \end{pmatrix}$$

$\langle v_i, v_j \rangle$ is the inner product of v_i and v_j : $\langle v_i, v_j \rangle = v_i \cdot v_j = \|v_i\| \cdot \|v_j\| \cdot \cos(\theta_{i,j})$. $\theta_{i,j}$ is the angle between v_i and v_j .

If the vectors have all a (Euclidean) norm of one, $\|v_i\| = \|v_j\| = 1$, then we obtain a matrix of angles between vectors $\theta_{i,j}$.

Gramian Angular Field (GAF)

The standard gram matrix captures linear correlations between elements in a time series and may fail to capture richer representations of the latent states that drive the series.

Wang and Oates (2015) suggest an encoding method from a polar coordinates transformation using the following two steps:

1. Normalize a time series $\{x_1, \dots, x_T\}$ between -1 and 1 (Min-Max scaler). Denote by $\tilde{x}_t$ the re-scaled version of observation x_t .
2. Convert the normalized data into “polar coordinates”: $\phi_t = \arccos(\tilde{x}_t)$.

We then build a gram-like matrix:

$$G = \begin{pmatrix} \cos(\phi_1 + \phi_1) & \cos(\phi_1 + \phi_2) & \cdots & \cos(\phi_1 + \phi_T) \\ \cos(\phi_2 + \phi_1) & \cos(\phi_2 + \phi_2) & \cdots & \cos(\phi_2 + \phi_T) \\ \vdots & \vdots & \ddots & \vdots \\ \cos(\phi_T + \phi_1) & \cos(\phi_T + \phi_2) & \cdots & \cos(\phi_T + \phi_T) \end{pmatrix}$$

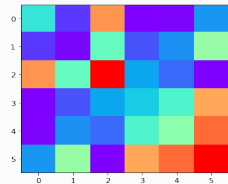
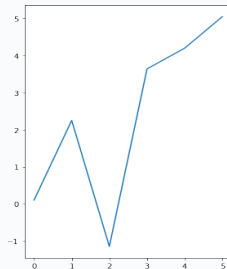
which will serve as a 2-dimensional version of the time series.

Simple example of GAF

The GAF representation of the time series penalizes those pairs of observations that are close to the mean.

Consider the time series $\{0.1, 2.25, -1.15, 3.64, 4.2, 5.05\}$. We can perform the GAF transformation as we show below:

```
# Array
X = np.array([0.1, 2.25, -1.15, 3.64, 4.2, 5.05])
# Min-Max scaling
X_norm = (X - np.min(X))/(np.max(X)-np.min(X))
X_norm = X_norm * 2 - 1
# Polar transformation
X_arccos = np.arccos(X_norm)
X_arccos_m = np.add.outer(X_arccos, X_arccos)
GramAngular = np.cos(X_arccos_m)
```



```
fig, ax = plt.subplots(1, 2, figsize=(15,7), gridspec_kw={'width_ratios': [1, 1]})
ax[0].plot(X)
ax[1].imshow(GramAngular, cmap='rainbow')
```

Summary of Lesson 3

In Lesson 3, we have:

- ▶ Shown that time series prediction problems can be transformed into image recognition tasks using convolutional neural networks.
- ▶ Described Gramian Angular Field as a tool to transform time series into images.

⇒ **References for this lesson:**

Wang, Zhiguang, and Tim Oates. "Encoding Time Series as Images for Visual Inspection and Classification Using Tiled Convolutional Neural Networks." In *Workshops at the Twenty-Ninth AAAI Conference on Artificial Intelligence*. 2015.

de Vitry, Louis. "Encoding Time Series as Images." *Medium*. October 2018.

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement the GAF imaging for financial data.

Module 3: Lesson 4

Deep Convolutional Neural Networks: Implementation of Gramian Angular Field



Outline

- ▶ Setting up the data
- ▶ GAF for financial time series

Setting up the data

In this lesson, we implement the Gramian Angular Field (GAF) methodology that we described in Lesson 3 and illustrate its outcome for a general financial time series.

To apply GAF, we need to rearrange our time series into a 2-dimensional array where each row represents a sub-sequence of the whole time series.

Formally, we split a time series $\{x_1, x_2, \dots, x_K\}$ into sequences of T observations $\{x_t, x_{t+1}, x_{t+2}, \dots, x_{t+T}\}$, for $t \in [1, K - T]$. Then, we form the array X that stacks in rows each of these sequences:

$$X = \begin{pmatrix} x_1 & x_2 & \cdots & x_T \\ x_2 & x_3 & \cdots & x_{T+1} \\ x_3 & x_4 & \cdots & x_{T+2} \\ \vdots & \vdots & \ddots & \vdots \\ x_{K-T} & x_{K-T+1} & \cdots & x_K \end{pmatrix}$$

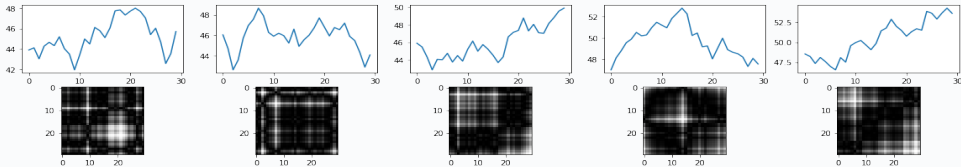
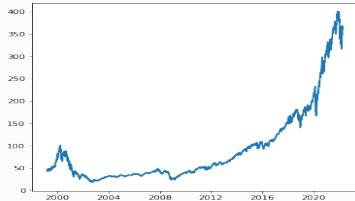
For each time series in the array above, we separately apply the GAF transformation.

GAF transformation

We implement the GAF transformation to the time series of Nasdaq prices (QQQ).

We slice the time series into sequences of 30 days and obtain the GAF image for each of them.

From the GAF representation, we train a CNN to predict the returns from Nasdaq.



Summary of Lesson 4

In Lesson 4, we have:

- ▶ Implemented GAF imaging to transform time series into images.
- ▶ Used the outputs of GAF as inputs for CNN architectures to predict stock returns.

⇒ **References for this lesson:**

Wang, Zhiguang, and Tim Oates. "Encoding Time Series as Images for Visual Inspection and Classification Using Tiled Convolutional Neural Networks." In *Workshops at the Twenty-Ninth AAAI Conference on Artificial Intelligence*. 2015.

de Vitry, Louis. "Encoding Time Series as Images." *Medium*. October 2018.

TO DO NEXT: Please go to the Jupyter Notebook associated with Lesson 4.

With this, we complete the descriptions related to convolutional neural networks. The next two modules are devoted to time series deep learning models. See you there!

```
from datetime import date
try:
    import yfinance as yf
except:
    !pip install yfinance
    import yfinance as yf
import pandas as pd
```

Looking in indexes: <https://pypi.org/simple>, <https://us-python.pkg.dev/colab-wheels/public/simple/>
Collecting yfinance
 Downloading yfinance-0.2.12-py2.py3-none-any.whl (59 kB)
 59.2/59.2 KB 1.8 MB/s eta 0:00:00
Requirement already satisfied: pandas>=1.3.0 in /usr/local/lib/python3.9/dist-packages (from yfinance) (1.4.4)
Collecting frozendict>=2.3.4
 Downloading frozendict-2.3.5-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (112 kB)
 112.8/112.8 KB 5.6 MB/s eta 0:00:00
Collecting appdirs>=1.4.4
 Downloading appdirs-1.4.4-py2.py3-none-any.whl (9.6 kB)
Requirement already satisfied: pytz>=2022.5 in /usr/local/lib/python3.9/dist-packages (from yfinance) (2022.7.1)
Requirement already satisfied: multitasking>=0.0.7 in /usr/local/lib/python3.9/dist-packages (from yfinance) (0.0.11)
Collecting beautifulsoup4>=4.11.1
 Downloading beautifulsoup4-4.11.2-py3-none-any.whl (129 kB)
 129.4/129.4 KB 8.7 MB/s eta 0:00:00
Collecting cryptography>=3.3.2
 Downloading cryptography-39.0.2-cp36-abi3-manylinux_2_28_x86_64.whl (4.2 MB)
 4.2/4.2 MB 33.9 MB/s eta 0:00:00
Collecting html5lib>=1.1
 Downloading html5lib-1.1-py2.py3-none-any.whl (112 kB)
 112.2/112.2 KB 3.6 MB/s eta 0:00:00
Requirement already satisfied: lxml>=4.9.1 in /usr/local/lib/python3.9/dist-packages (from yfinance) (4.9.2)
Requirement already satisfied: numpy>=1.16.5 in /usr/local/lib/python3.9/dist-packages (from yfinance) (1.22.4)
Collecting requests>=2.26
 Downloading requests-2.28.2-py3-none-any.whl (62 kB)
 62.8/62.8 KB 4.4 MB/s eta 0:00:00
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.9/dist-packages (from beautifulsoup4>=4.11.1->yfinance) (1.0.1)
Requirement already satisfied: cffi>=1.12 in /usr/local/lib/python3.9/dist-packages (from cryptography>=3.3.2->yfinance) (1.15.0)
Requirement already satisfied: six>=1.9 in /usr/local/lib/python3.9/dist-packages (from html5lib>=1.1->yfinance) (1.15.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.9/dist-packages (from html5lib>=1.1->yfinance) (0.5.1)
Requirement already satisfied: python-dateutil>=2.8.1 in /usr/local/lib/python3.9/dist-packages (from pandas>=1.3.0->yfinance) (2.8.1)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.9/dist-packages (from requests>=2.26->yfinance) (2022.9.24)
Requirement already satisfied: urllib3<1.27,>=1.21.1 in /usr/local/lib/python3.9/dist-packages (from requests>=2.26->yfinance) (1.26.15)
Collecting charset-normalizer<4,>=2
 Downloading charset-normalizer-3.1.0-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (199 kB)
 199.2/199.2 KB 10.1 MB/s eta 0:00:00
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.9/dist-packages (from requests>=2.26->yfinance) (2.10)
Requirement already satisfied: pyparsing in /usr/local/lib/python3.9/dist-packages (from cffi>=1.12->cryptography>=3.3.2->yfinance) (3.0.9)
Installing collected packages: appdirs, html5lib, frozendict, charset-normalizer, beautifulsoup4, requests, cryptography, yfinance
 Attempting uninstall: html5lib
 Found existing installation: html5lib 1.0.1
 Uninstalling html5lib-1.0.1:
 Successfully uninstalled html5lib-1.0.1
 Attempting uninstall: beautifulsoup4
 Found existing installation: beautifulsoup4 4.9.3
 Uninstalling beautifulsoup4-4.9.3:
 Successfully uninstalled beautifulsoup4-4.9.3
 Attempting uninstall: requests
 Found existing installation: requests 2.25.1
 Uninstalling requests-2.25.1:
 Successfully uninstalled requests-2.25.1
Successfully installed appdirs-1.4.4 beautifulsoup4-4.11.2 charset-normalizer-3.1.0 cryptography-39.0.2 frozendict-2.3.5 htm

```
from google.colab import drive
drive.mount('/content/drive')
drivepath = "/content/drive/MyDrive/WQU/WQU - Deep Learning/DL - Module 3/Candle Data"
```

Mounted at /content/drive

DEEP LEARNING FOR FINANCE

MODULE 3 | LESSON 2

✓ TRANSFER LEARNING FOR CONVOLUTIONAL NEURAL NETWORKS

Reading Time	45 minutes
Prior Knowledge	CNNs, MLPs, Machine learning
Keywords	Transfer learning, convolutional networks

1. INTRODUCTION

In previous lessons, we described a series of CNN architectures that have been proven successful in dealing with complex image recognition tasks. Most of the time, these architectures are large-scale models trained from large-scale datasets.

We can leverage these trained architectures in our own classification problems. This technique, labeled **transfer learning**, may solve two issues. First, we may not have many training examples, which may lead to overfitting of complicated models. Second, the accuracy of the trained model with relatively few examples may not meet practical requirements.

Transfer learning applies the knowledge learned from a *source* dataset where the network was trained to a *target* dataset. Although the images in our target dataset may be completely different from those contained in the source dataset, the transferred model may be able to extract more general features. This ability may turn out to also be effective for recognizing the objects in our target dataset. Thus, transfer learning may considerably enhance the predicted ability of the model using a pre-trained model with a relatively low computational cost.

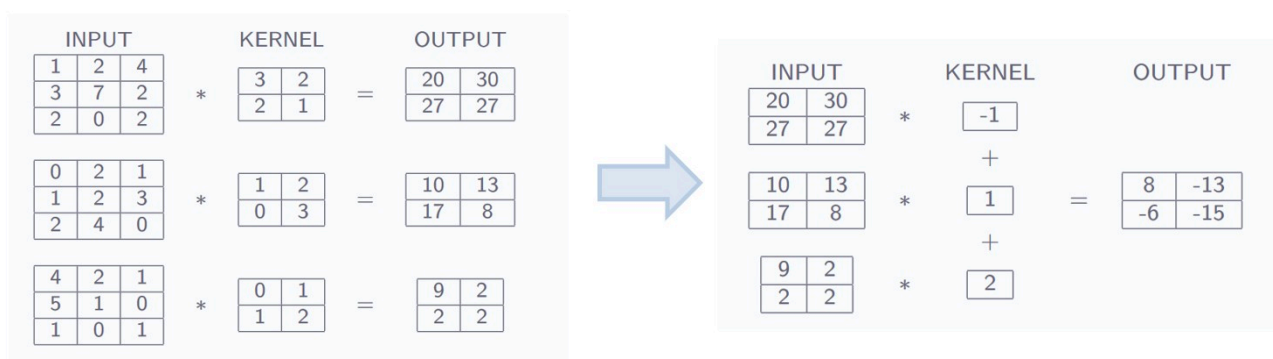
Transfer learning involves the following steps:

1. Pre-train a neural network model, i.e., the source model, on a source dataset. We can also obtain the pre-trained parameters directly as obtained from an external source.
2. Create a new neural network model, i.e., the target model. This copies the architecture and the parameters on the source model except the output layer. The parameters contain the knowledge learned from the source dataset, and this knowledge will also be applicable to the target dataset. We do not use the pre-trained output layer because the output layer of the source model is closely related to the labels of the source dataset.
3. Add an output layer to the target model, whose number of outputs is the number of categories in the target dataset. Then, randomly initialize the model parameters of this layer.
4. Train the target model on the target dataset, such as our candlestick charts. The output layer will be trained from scratch, while the parameters of all the other layers are either left untrained and fixed to their pre-trained values or fine-tuned with initial values set at those of the source model and using a very small learning rate.

In this notebook, we are going to use transfer learning to train a CNN to predict stock returns from candlestick charts. Our pre-trained architecture is called "[Xception](#)." The Xception architecture exploits a special type of convolutional operator called "Separable convolution," which is meant to reduce the computational burden of the model and generalizes the multi-branch blocks that are included in the "Inception" model.

A separable convolution is composed of two steps. First, we apply the usual convolution operator to each input channels using a single kernel with as many dimensions as input channels. However, instead of adding up the results as in a standard convolution, we keep the results separate in different dimensions (that is why this convolution is called "Separable").

In the second step, we apply 1x1 convolution to the layer obtained in the previous step, with as many output channels as we desire, using the standard convolution operator. That is, we have, for each output dimension, a 1x1 kernel with as many dimensions as input channels.



To get some idea of the savings of computational costs, consider the following example. Assume we have an input image with 256x256 pixels and 3 channels. Consider a standard convolutional layer that applies kernels of width equal to 3, with 64 output channels, no padding, and a stride of 1. Thus, the output's height and width will be of 254. In a single kernel window, the convolution implies 9 multiplications. This means that sliding a single kernel through the whole input requires $9 \times 254 \times 254 \times 3$ multiplications, or 1,741,932. Repeating this through all the output channels, 64, increases the multiplications to 111,483,648.

Now let's consider the computational requirements of a separable convolution. The first step requires the initial $9 \times 254 \times 254 \times 3$ multiplications, and the second step, 1x1 convolutions, involves $254 \times 254 \times 3 \times 64$ additional multiplications. In total, the separable

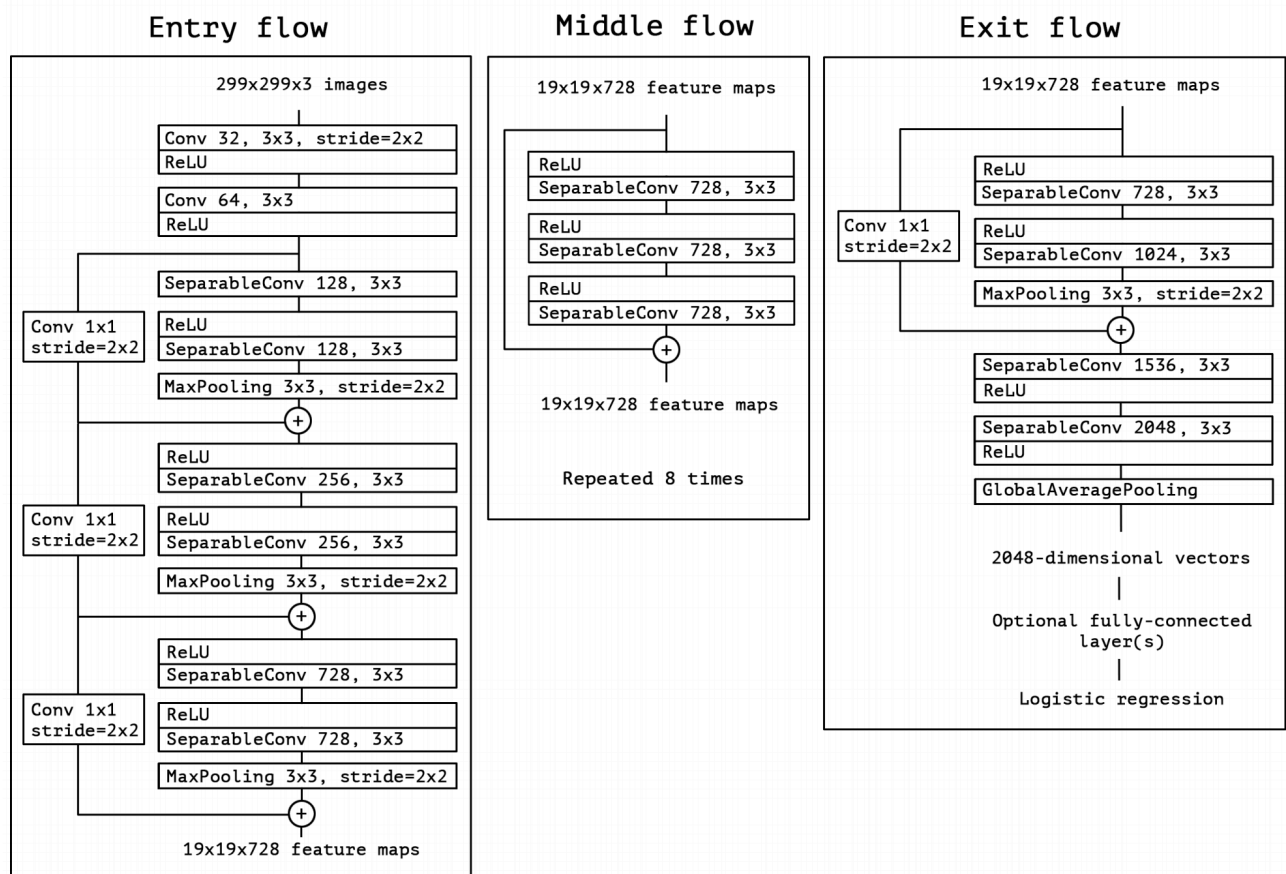
convolution makes 14,129,004 multiplications. These are a great deal of multiplications but a lot less than those performed with a standard convolution.

The separable convolution considerably reduces the number of parameters in a convolution. In the example above, the separable convolution involves 27 weights in the 3x3 kernels and 192 weights in the 1x1 convolutions. In contrast, the standard convolution would require $9 \times 3 \times 64 = 1,728$ trainable parameters. This may damage your network if it is already small. If used properly, separable convolutions may speed up training without damaging the predictive ability.

You can find extended descriptions of separable convolutions in the following [link](#).

As illustrated below, Xception is a CNN architecture that contains successive layers of multi-branch blocks of convolutions, where the "inside" branches includes separable convolutions.

[Source of picture](#).



2. LOADING THE DATASET

```
import tensorflow as tf
path_train = drivepath + '/Train'
path_test = drivepath + '/Test'
# Training set
train = tf.keras.preprocessing.image_dataset_from_directory(path_train,
                                                            image_size=(250, 250),
                                                            label_mode='binary')

# Validation set
test = tf.keras.preprocessing.image_dataset_from_directory(path_test,
                                                           image_size=(250, 250),
                                                           label_mode='binary')

classes = train.class_names
```

Found 1280 files belonging to 2 classes.
Found 360 files belonging to 2 classes.

```
# Transform the input images to numpy arrays of dimension 250x250x3
import numpy as np
X_train, y_train = np.array([]).reshape(-1, 250, 250, 3), np.array([]).reshape(-1, 1)
for X, y in train:
    X_train = np.concatenate([X_train, X])
    y_train = np.concatenate([y_train, y])
```

3. ARCHITECTURE

As described above, we are going to train a CNN using transfer learning. We load the pre-trained Xception network. We refer to the documentation available [here](#), where you can see the available options.

The pretrained Xception architecture represents the main body of the network. We just need to determine the shape of the input tensor (250x250x3) and then add the "top" part of the network that represents our trainable part of the model. We include a flattening layer and a fully connected layer with 512 hidden units and ReLU activation, which then leads to a final output layer with a sigmoid activation.

```
base = tf.keras.applications.Xception(
    input_tensor=tf.keras.layers.Input(shape=(250,250,3)),
    weights='imagenet',
    include_top=False,
    pooling = 'avg'
)
flatten = tf.keras.layers.Flatten()(base.output)
fcdense = tf.keras.layers.Dense(512, activation='relu')(flatten)
sigmoid = tf.keras.layers.Dense(1, activation='sigmoid')(fcdense)
model = tf.keras.models.Model(inputs=base.input, outputs=sigmoid)
model.compile(loss='binary_crossentropy', optimizer='RMSprop', metrics=['accuracy'])
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/xception/xception_weights_tf_dim_ordering
83683744/83683744 [=====] - 4s 0us/step

Let's see the structure of the model, which is gigantic.

```
model.summary()
```

Model: "model"

Layer (type)	Output Shape	Param #	Connected to
=====			
input_1 (InputLayer)	[(None, 250, 250, 3)]	0	[]
block1_conv1 (Conv2D)	(None, 124, 124, 32)	864	['input_1[0][0]']
block1_conv1_bn (BatchNormalization)	(None, 124, 124, 32)	128	['block1_conv1[0][0]']
block1_conv1_act (Activation)	(None, 124, 124, 32)	0	['block1_conv1_bn[0][0]']
block1_conv2 (Conv2D)	(None, 122, 122, 64)	18432	['block1_conv1_act[0][0]']
block1_conv2_bn (BatchNormalization)	(None, 122, 122, 64)	256	['block1_conv2[0][0]']
block1_conv2_act (Activation)	(None, 122, 122, 64)	0	['block1_conv2_bn[0][0]']
block2_sepconv1 (SeparableConv2D)	(None, 122, 122, 128)	8768	['block1_conv2_act[0][0]']
block2_sepconv1_bn (BatchNormalization)	(None, 122, 122, 128)	512	['block2_sepconv1[0][0]']
block2_sepconv2_act (Activation)	(None, 122, 122, 128)	0	['block2_sepconv1_bn[0][0]']
block2_sepconv2 (SeparableConv2D)	(None, 122, 122, 128)	17536	['block2_sepconv2_act[0][0]']
block2_sepconv2_bn (BatchNormalization)	(None, 122, 122, 128)	512	['block2_sepconv2[0][0]']
conv2d (Conv2D)	(None, 61, 61, 128)	8192	['block1_conv2_act[0][0]']
block2_pool (MaxPooling2D)	(None, 61, 61, 128)	0	['block2_sepconv2_bn[0][0]']
batch_normalization (BatchNormalization)	(None, 61, 61, 128)	512	['conv2d[0][0]']
add (Add)	(None, 61, 61, 128)	0	['block2_pool[0][0]', 'batch_normalization[0][0]']
block3_sepconv1_act (Activation)	(None, 61, 61, 128)	0	['add[0][0]']

```

block3_sepconv1 (SeparableConv (None, 61, 61, 256) 33920 ['block3_sepconv1_act[0][0]']
2D)
block3_sepconv1_bn (BatchNorm (None, 61, 61, 256) 1024 ['block3_sepconv1[0][0]']

```

3. TRAINING AND PERFORMANCE

Now, we train the model for 20 epochs, giving some additional weight to the class that is less frequent in the dataset. We will choose the model that achieves the highest accuracy in the validation set.

```

checkpoint = tf.keras.callbacks.ModelCheckpoint('model.h5', monitor='val_accuracy', save_best_only=True, mode='auto')
from sklearn.utils import class_weight
class_weight = {0: 1/(1-np.mean(y_train)), 1: 1.0}
print(class_weight)
model.fit(train, epochs=20, validation_data=test, verbose=1, callbacks=[checkpoint], class_weight = class_weight)

{0: 1.5274463007159904, 1: 1.0}
Epoch 1/20
40/40 [=====] - 200s 4s/step - loss: 0.8583 - accuracy: 0.6461 - val_loss: 0.6988 - val_accuracy: 0.6461
Epoch 2/20
40/40 [=====] - 29s 701ms/step - loss: 0.7289 - accuracy: 0.6547 - val_loss: 0.7165 - val_accuracy: 0.6547
Epoch 3/20
40/40 [=====] - 28s 680ms/step - loss: 0.6243 - accuracy: 0.6609 - val_loss: 2.4851 - val_accuracy: 0.6609
Epoch 4/20
40/40 [=====] - 28s 672ms/step - loss: 0.5227 - accuracy: 0.7641 - val_loss: 3.7885 - val_accuracy: 0.7641
Epoch 5/20
40/40 [=====] - 32s 770ms/step - loss: 0.4211 - accuracy: 0.8344 - val_loss: 4.8579 - val_accuracy: 0.8344
Epoch 6/20
40/40 [=====] - 29s 696ms/step - loss: 0.2991 - accuracy: 0.8891 - val_loss: 3.9779 - val_accuracy: 0.8891
Epoch 7/20
40/40 [=====] - 28s 684ms/step - loss: 0.2284 - accuracy: 0.9258 - val_loss: 4.1823 - val_accuracy: 0.9258
Epoch 8/20
40/40 [=====] - 29s 702ms/step - loss: 0.1716 - accuracy: 0.9500 - val_loss: 3.1386 - val_accuracy: 0.9500
Epoch 9/20
40/40 [=====] - 29s 692ms/step - loss: 0.1261 - accuracy: 0.9578 - val_loss: 2.4352 - val_accuracy: 0.9578
Epoch 10/20
40/40 [=====] - 29s 698ms/step - loss: 0.1069 - accuracy: 0.9672 - val_loss: 6.0182 - val_accuracy: 0.9672
Epoch 11/20
40/40 [=====] - 31s 748ms/step - loss: 0.0888 - accuracy: 0.9742 - val_loss: 11.5098 - val_accuracy: 0.9742
Epoch 12/20
40/40 [=====] - 30s 696ms/step - loss: 0.1024 - accuracy: 0.9695 - val_loss: 2.4064 - val_accuracy: 0.9695
Epoch 13/20
40/40 [=====] - 30s 714ms/step - loss: 0.0622 - accuracy: 0.9844 - val_loss: 12.3980 - val_accuracy: 0.9844
Epoch 14/20
40/40 [=====] - 30s 734ms/step - loss: 0.0786 - accuracy: 0.9766 - val_loss: 3.0915 - val_accuracy: 0.9766
Epoch 15/20
40/40 [=====] - 28s 688ms/step - loss: 0.0600 - accuracy: 0.9828 - val_loss: 2.8822 - val_accuracy: 0.9828
Epoch 16/20
40/40 [=====] - 28s 677ms/step - loss: 0.0662 - accuracy: 0.9820 - val_loss: 4.1083 - val_accuracy: 0.9820
Epoch 17/20
40/40 [=====] - 28s 690ms/step - loss: 0.0425 - accuracy: 0.9891 - val_loss: 4.1419 - val_accuracy: 0.9891
Epoch 18/20
40/40 [=====] - 29s 690ms/step - loss: 0.0512 - accuracy: 0.9859 - val_loss: 3.5634 - val_accuracy: 0.9859
Epoch 19/20
40/40 [=====] - 29s 700ms/step - loss: 0.0921 - accuracy: 0.9805 - val_loss: 3.8272 - val_accuracy: 0.9805
Epoch 20/20
40/40 [=====] - 29s 693ms/step - loss: 0.0605 - accuracy: 0.9812 - val_loss: 2.0401 - val_accuracy: 0.9812
<keras.callbacks.History at 0x7f7ee808f040>

```

Below we present the performance metrics of the trained model. The performance of the model depends on the random initialization of the trainable parameters. However, if you train the model several times, you will notice that it tends to perform better than the basic CNN architecture that we trained at the end of Module 2, achieving some gains in accuracy and other metrics. Notice, however, that the model still seems to predict too often a "down" move. Bear in mind, that the number of examples used in this application is quite small relative to the datasets that we usually confront in image recognition applications. You can see that the model overfits the training set, which may be solved by adding more examples in training that allows the model to improve its generalization capacity.

```

from sklearn import metrics
model = tf.keras.models.load_model('model.h5')
acc = model.evaluate(test)
print("Accuracy: {:.2f}%".format(acc[1] * 100))
X_test, y_test = np.array([]).reshape(-1, 250, 250, 3), np.array([]).reshape(-1, 1)
for X, y in test:
    X_test = np.concatenate([X_test, X])
    y_test = np.concatenate([y_test, y])
y_prob = model.predict(X_test)
y_pred = np.round(y_prob)
cm = metrics.confusion_matrix(y_test, y_pred)
cm = metrics.ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=classes)
cm.plot(values_format='', cmap='Blues')

```

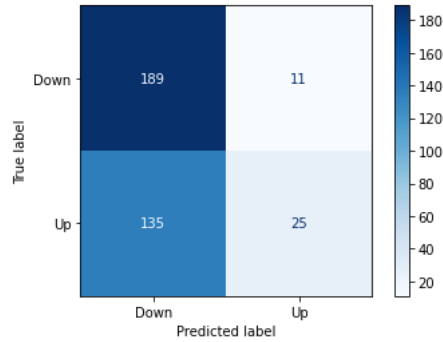
```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

```
12/12 [=====] - 4s 159ms/step - loss: 3.8272 - accuracy: 0.5944
```

```
Accuracy: 59.44%
```

```
12/12 [=====] - 3s 171ms/step
```

	precision	recall	f1-score	support
0.0	0.58	0.94	0.72	200
1.0	0.69	0.16	0.26	160
accuracy			0.59	360
macro avg	0.64	0.55	0.49	360
weighted avg	0.63	0.59	0.51	360



4. CONCLUSION

In this lesson, we have described how transfer learning may enhance the performance of our CNNs while reducing the computational costs. We have also introduced the concept of "separable convolution" that is used in the Xception architecture.